Metody Inteligencji Obliczeniowej

Sprawozdanie 4

Wydział Fizyki i Informatyki Stosowanej

Mikhail Shupliakou

13.04.2026

Spis treści

1 Wstęp **3**
1.1 Definicja zbioru rozmytego 3
1.2 Zastosowania 3

2 Zadanie 1 **3**
2.1 Opis i cel zadania 3
2.2 Implementacja i konfiguracja systemu rozmytego 4
2.3 Definicja zmiennych lingwistycznych i zakresów 4
2.4 Zastosowanie gaussowskich funkcji przynależności 4
2.5 Wnioskowanie i wizualizacja 5
2.6 Tworzenie reguł 5
2.7 Weryfikacja działania systemu i symulacja 6
2.8 Przebieg symulacji 6
2.9 Wynik Symulacji 6
2.10 Analiza powierzchni sterującej – Heatmapa 7
2.11 Po co powstała ta wizualizacja? 7
2.12 Interpretacja wyników 8
2.13 Podsumowanie i wnioski 8

3 Zadanie 2 **9**
3.1 Opis i cel zadania 9
3.2 Implementacja systemu oceny studenta 9
3.3 Proces projektowania systemu 9
3.4 Przebieg wnioskowania i wynik symulacji 9
3.5 Interpretacja wizualizacji 10

4 Spis Literatury i odpowiednie linki **11**

Zbiory rozmyte

1 Wstęp

1.1 Definicja zbioru rozmytego

Teoria zbiorów rozmytych, wprowadzona przez Lotfiego Zadeha w 1965 roku, stanowi rozszerzenie klasycznej teorii zbiorów. W tradycyjnej logice dwuwartościowej element albo należy do zbioru (1), albo do niego nie należy (0). Zbiór rozmyty pozwala na stopniowanie przynależności.

Zbiór rozmyty A w pewnej przestrzeni rozważań X (uniwersum) definiujemy jako zbiór par uporządkowanych:

$$A = \{(x, \mu_A(x)) : x \in X\} \tag{1}$$

gdzie $\mu_A(x)\colon X \to [0, 1]$ jest funkcją przynależności. Wartość $\mu_A(x)$ określa stopień przynależności elementu x do zbioru A:

- $\mu_A(x) = 1$ oznacza pełną przynależność,
- $\mu_A(x) = 0$ oznacza całkowity brak przynależności,
- $0 < \mu_A(x) < 1$ oznacza częściową przynależność.

1.2 Zastosowania

Zbiory rozmyte znajdują szerokie zastosowanie w systemach sterowania (Fuzzy Control), procesach decyzyjnych oraz w modelowaniu pojęć nieostrych, które występują w języku naturalnym (np. "wysoka temperatura", "mała prędkość"). Pozwalają one na budowanie algorytmów odpornych na nieprecyzyjne dane wejściowe.

2 Zadanie 1

2.1 Opis i cel zadania

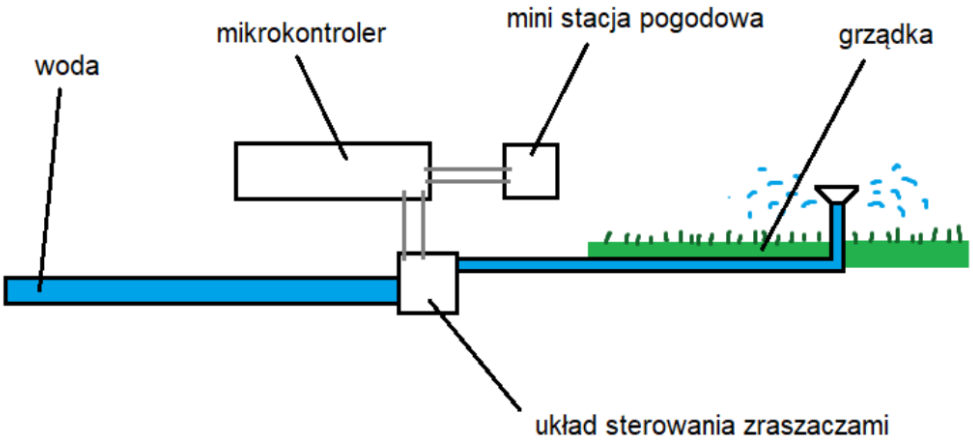
Celem zadania jest zaprojektowanie rozmytego systemu sterowania (Fuzzy Control System) dla automatycznego procesu podlewania grządki o powierzchni $3m^2$. System ma za zadanie optymalizować zużycie wody w zależności od warunków atmosferycznych, przyjmując maksymalną wydajność na poziomie 25 l/dobę.

Dane wejściowe systemu:

- **Wilgotność (H):** mierzona przez higrometr w zakresie $[0, 100]\%$.
- **Temperatura (T):** mierzona w ciągu dnia w zakresie $[15, 35]°C$.

Dane wyjściowe:

- **Ilość wody (Q):** dzienna dawka nawadniania w zakresie $[0, 25]$ litrów.



Rysunek 1: Schemat systemu sterowania dla auto podlewania

2.2 Implementacja i konfiguracja systemu rozmytego

W celu realizacji zadania wykorzystano bibliotekę `scikit-fuzzy` w środowisku Python. System został zaprojektowany jako model sterowania typu Mamdaniego, składający się z dwóch wejść (antecedents) oraz jednego wyjścia (consequent).

2.3 Definicja zmiennych lingwistycznych i zakresów

Zgodnie z założeniami projektowymi, zdefiniowano następujące uniwersa rozważań:

- **Temperatura:** zakres $[15, 35]°C$ z krokiem 0.1.
- **Wilgotność:** zakres $[0, 100]\%$ z krokiem 0.1.
- **Ilość wody (podlewanie):** zakres $[0, 25]$ litrów na dobę. Rozszerzenie dolnej granicy do 0 pozwala na całkowite wstrzymanie podlewania w warunkach wysokiej wilgotności.

```
temperature = ctrl.Antecedent(np.arange(15, 35.1, 0.1), 'temperature')
wetness = ctrl.Antecedent(np.arange(0, 100.1, 0.1), 'wetness')
watering = ctrl.Consequent(np.arange(0, 25.1, 0.1), 'watering')
```

2.4 Zastosowanie gaussowskich funkcji przynależności

W projekcie zdecydowano się na odejście od klasycznych funkcji trójkątnych na rzecz **gaussowskich funkcji przynależności** (*Gaussian Membership Functions*). Wybór ten jest podyktowany chęcią uzyskania płynnych przejść między stanami systemu, co lepiej oddaje procesy fizyczne i biologiczne zachodzące w glebie.

$$f(x; \sigma, c) = e^{\frac{-(x-c)^2}{2\sigma^2}} \tag{2}$$

Konfiguracja zbiorów rozmytych:

- **Temperatura:** Centra funkcji ustalono w punktach 15 (cool), 25 (warm) oraz 35 (hot) przy odchyleniu standardowym $\sigma = 4$. Zapewnia to optymalne pokrycie całego zakresu pracy higrometru.

- **Wilgotność:** Centra ustalono w punktach 0 (dry), 50 (norm) i 100 (wet) przy $\sigma = 20$, co pozwala na łagodną zmianę decyzji sterownika przy wahaniach wilgotności gleby.
- **Podlewanie:** Zbiory wyjściowe (*little, more, a lot*) rozmieszczono symetrycznie w zakresie $[0, 25]$ z centrami odpowiednio $0, 12.5$ oraz 25 litrów ($\sigma = 5$). Dzięki temu system jest w stanie precyzyjnie dozować wodę w okolicach zalecanej normy 10 l/dobę.

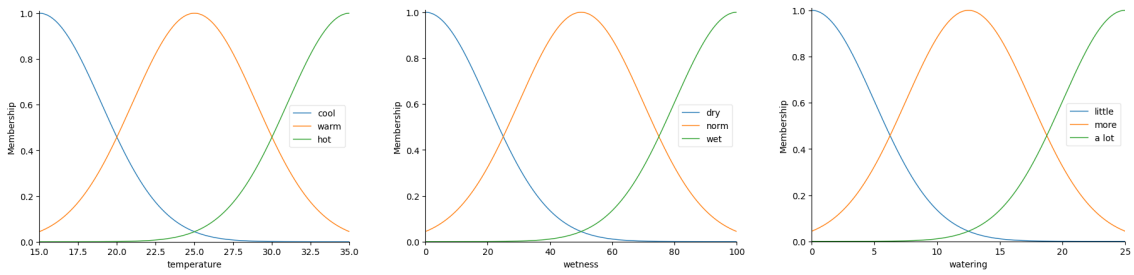
```
temperature['cool'] = fuzz.gaussmf(temperature.universe, 15, 4)
temperature['warm'] = fuzz.gaussmf(temperature.universe, 25, 4)
temperature['hot'] = fuzz.gaussmf(temperature.universe, 35, 4)

wetness['dry'] = fuzz.gaussmf(wetness.universe, 0, 20)
wetness['norm'] = fuzz.gaussmf(wetness.universe, 50, 20)
wetness['wet'] = fuzz.gaussmf(wetness.universe, 100, 20)

watering['little'] = fuzz.gaussmf(watering.universe, 0, 5)
watering['more'] = fuzz.gaussmf(watering.universe, 12.5, 5)
watering['a lot'] = fuzz.gaussmf(watering.universe, 25, 5)
```

2.5 Wnioskowanie i wizualizacja

Zastosowanie funkcji Gaussa pozwoliło na wygenerowanie gładkiej powierzchni sterującej (*control surface*). W przeciwieństwie do funkcji trapezoidalnych, system oparty na rozkładzie Gaussa reaguje na każdą, nawet najmniejszą zmianę parametrów wejściowych, co zapobiega gwałtownym skokom wydajności pompy nawadniającej i sprzyja oszczędności zasobów wodnych.



(a) Funkcje przynależności - Temperatura

(b) Funkcje przynależności - Wilgotność

(c) Funkcje przynależności - Podlewanie

Rysunek 2: Wizualizacja zbiorów rozmytych dla systemu sterowania

2.6 Tworzenie reguł

W ramach implementacji sformułowano bazę dziewięciu reguł wnioskowania, które definiują logiczne powiązania między warunkami atmosferycznymi a intensywnością nawadniania grządki. Wszystkie reguły zostały zintegrowane w obiekcie `ControlSystem`, tworząc strukturę decyzyjną sterownika, a następnie zainicjowano moduł `ControlSystemSimulation`. Taka konfiguracja pozwala na praktyczne wykorzystanie modelu do przetwarzania danych wejściowych z czujników na konkretne wartości liczbowe, co umożliwia precyzyjne wyznaczenie dawki wody potrzebnej w danych warunkach temperatury i wilgotności.

```
# teraz stwórzmy zestaw zasad:

rule1 = ctrl.Rule(temperature['cool'] & wetness['dry'], watering['more'])
rule2 = ctrl.Rule(temperature['cool'] & wetness['norm'], watering['more'])
rule3 = ctrl.Rule(temperature['cool'] & wetness['wet'], watering['little'])

rule4 = ctrl.Rule(temperature['warm'] & wetness['dry'], watering['a lot'])
rule5 = ctrl.Rule(temperature['warm'] & wetness['norm'], watering['more'])
rule6 = ctrl.Rule(temperature['warm'] & wetness['wet'], watering['little'])

rule7 = ctrl.Rule(temperature['hot'] & wetness['dry'], watering['a lot'])
rule8 = ctrl.Rule(temperature['hot'] & wetness['norm'], watering['a lot'])
rule9 = ctrl.Rule(temperature['hot'] & wetness['wet'], watering['more'])

# Tworzymy system sterowania z podanych reguł
control_system = ctrl.ControlSystem([rule1, rule2, rule3, rule4, rule5, rule6,
↪  rule7, rule8, rule9])

# Tworzymy symulację systemu sterowania - tu będziemy podawać konkretne wartości
↪  wejściowe
model = ctrl.ControlSystemSimulation(control_system)
```

2.7 Weryfikacja działania systemu i symulacja

Następnym etapem prac było sprawdzenie poprawności działania zaprojektowanego systemu sterowania dla konkretnych wartości testowych. Proces ten pozwala na weryfikację, czy baza reguł oraz dobrane funkcje przynależności generują wyniki zgodne z oczekiwaniami agronomicznymi.

2.8 Przebieg symulacji

Do przeprowadzenia testu wykorzystano obiekt `ControlSystemSimulation`, do którego wprowadzono przykładowe dane pomiarowe:

- **Temperatura:** $20°C$ (warunki umiarkowane),
- **Wilgotność:** 10% (gleba sucha).

Po ustawieniu wejść wywołano metodę `model.compute()`, która uruchamia proces wnioskowania rozmytego. W tym kroku system analizuje stopień przynależności wejść do poszczególnych zbiorów, oblicza wagę każdej z dziewięciu reguł, a następnie dokonuje agregacji wyników.

```
# możemy już przejść do faktycznego wykorzystywania wyniku
model.input['temperature'] = 20
model.input['wetness'] = 10

# oblicz:
model.compute()

# wypisz wynik i przedstaw wynik graficznie:
print('Model proponuje liczbę litrów wody: ',(model.output['watering']))
watering.view(sim=model)
```

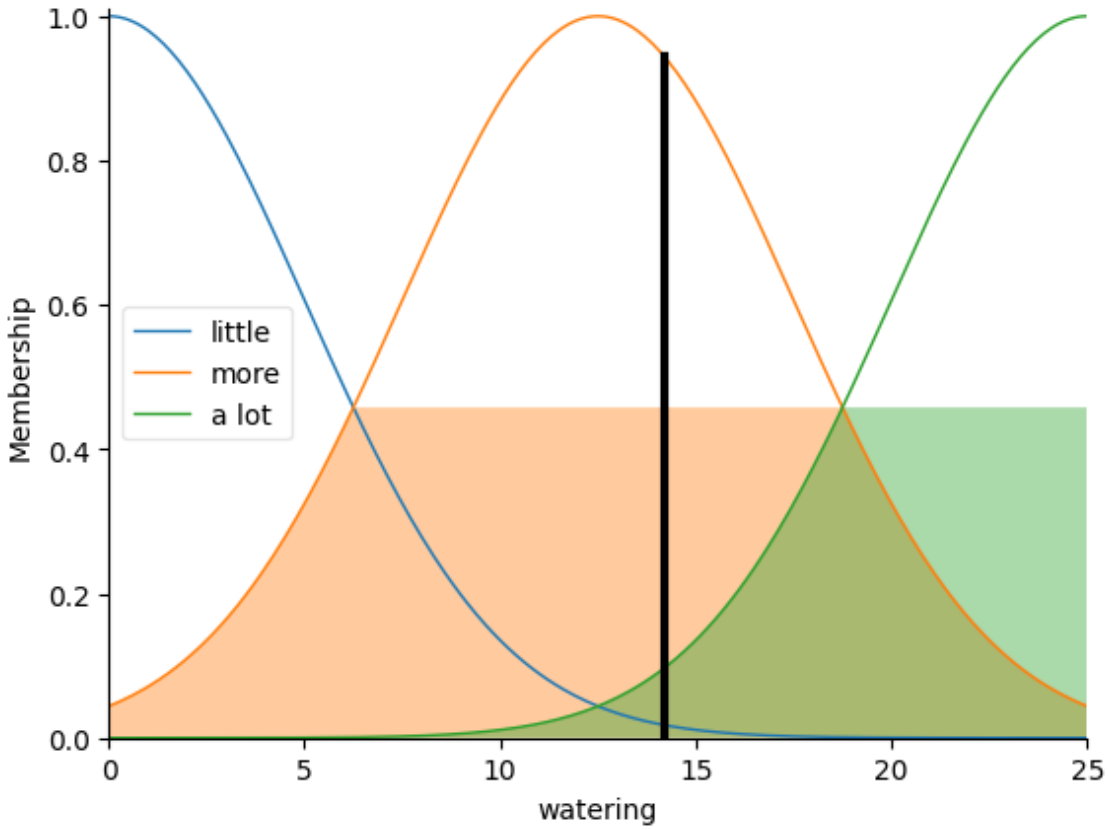
2.9 Wynik Symulacji

Otrzymany wynik rozmyty został poddany procesowi defuzyfikacji (standardowo metodą środka ciężkości — *centroid*), co pozwoliło na uzyskanie konkretnej warto-

ści liczbowej. Dla zadanych parametrów ($20°C, 10\%$), system zaproponował dawkę wody:

```
Model proponuje liczb litrw wody:  14.176174394112763
```

Wizualizacja graficzna za pomocą metody `watering.view(sim=model)` pozwala zaobserwować, jak rozkładają się wagi poszczególnych zbiorów wyjściowych (*little, more, a lot*) i w którym miejscu znajduje się środek ciężkości wypadkowej bryły geometrycznej, stanowiący ostateczną decyzję systemu.



Rysunek 3: Wagi poszczególnych zbiorów wyjściowych

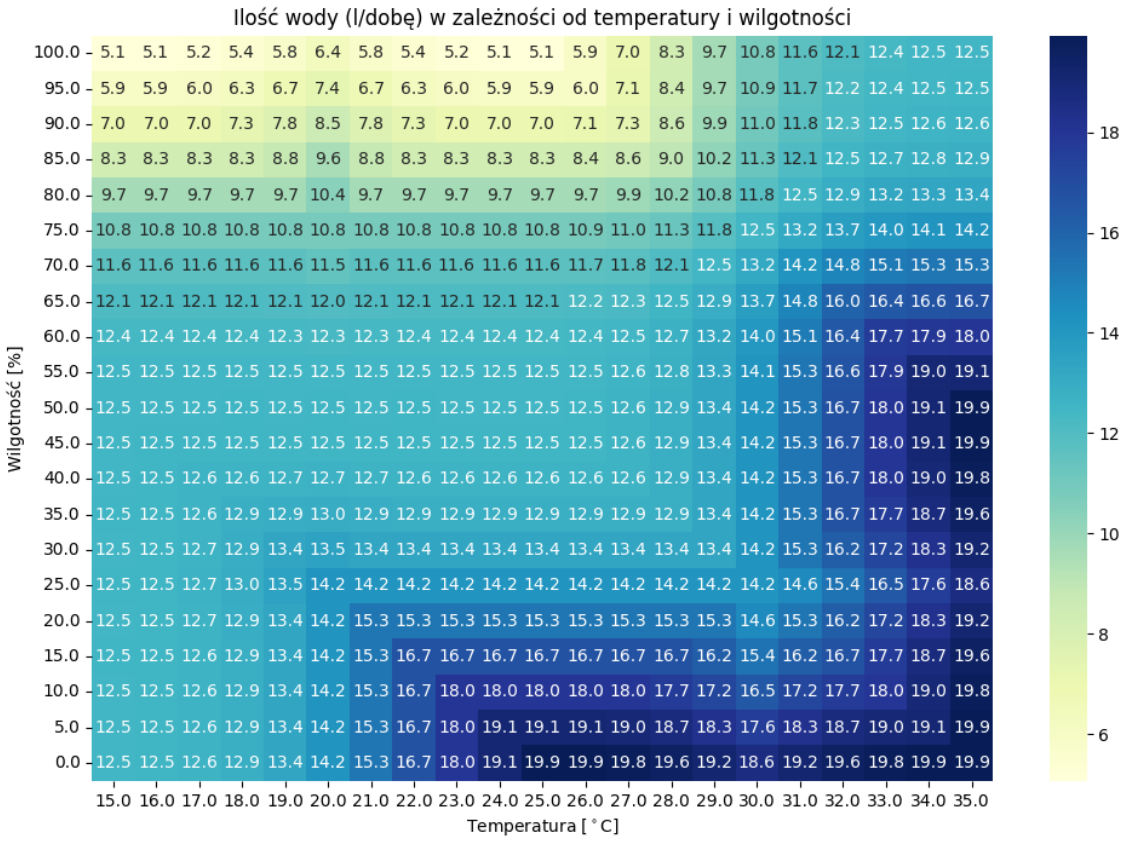
2.10 Analiza powierzchni sterującej – Heatmapa

Aby w pełni zrozumieć, jak zaprojektowany system zachowuje się w całym spektrum możliwych warunków, wygenerowałem mapę ciepła (*heatmap*). Jest to kluczowy element analizy, ponieważ pozwala on wyjść poza pojedyncze testy i spojrzeć na logikę sterownika "z lotu ptaka".

2.11 Po co powstała ta wizualizacja?

Pojedyncze symulacje (np. dla $20°C$) mówią nam tylko o konkretnym przypadku. Heatmapa natomiast pokazuje relację między temperaturą a wilgotnością dla każdego możliwego punktu w zdefiniowanym zakresie. Dzięki niej mogłem sprawdzić, czy w systemie nie występują "dziury"lub nielogiczne skoki wartości, które mogłyby świadczyć o błędnie sformułowanych regułach.

2.12 Interpretacja wyników



Rysunek 4: Heatmapa

Na wykresie (Rys. 4) wyraźnie widać płynne przejścia kolorów, co jest bezpośrednim efektem zastosowania funkcji Gaussa. Główne obserwacje to:

- **Maksymalne nawadnianie:** Zgodnie z oczekiwaniami, najciemniejsze (lub najbardziej intensywne) obszary znajdują się w prawym dolnym rogu, gdzie temperatura jest najwyższa, a wilgotność najniższa. Tu system zbliża się do wydajności 25 l/dobę.
- **Oszczędność wody:** W lewym górnym rogu (wysoka wilgotność i niska temperatura) wartości spadają do minimum. Pokazuje to, że sterownik "rozumie", iż w takich warunkach dodatkowe podlewanie jest zbędne.
- **Stabilność:** W środkowej części mapy wartości oscylują wokół zalecanych 10 litrów, co potwierdza, że system został dobrze skalibrowany pod kątem wymagań grządki o powierzchni $3m^2$.

Taka wizualizacja daje pewność, że system jest stabilny i gotowy do pracy w realnych, zmiennych warunkach pogodowych.

2.13 Podsumowanie i wnioski

Zaprojektowany system sterowania rozmytego dla automatycznego nawadniania grządki wykazał wysoką skuteczność w modelowaniu procesów zależnych od zmiennych danych atmosferycznych. Zastosowanie gaussowskich funkcji przynależności zamiast

standardowych modeli trójkątnych pozwoliło na uzyskanie ciągłej i gładkiej powierzchni sterującej, co w praktyce przekłada się na stabilną pracę układu bez gwałtownych skoków wydajności. Analiza wygenerowanej mapy ciepła potwierdziła, że system prawidłowo interpretuje zależności fizyczne: maksymalizuje dawkę wody (25 l/dobę) w warunkach wysokiej temperatury i niskiej wilgotności, jednocześnie drastycznie ograniczając zużycie zasobów w chłodne i deszczowe dni. Projekt dowodzi, że logika rozmyta jest optymalnym narzędziem do budowania inteligentnych systemów agro-technicznych, które skutecznie łączą dbałość o roślinność z efektywnym zarządzaniem zasobami wodnymi.

3 Zadanie 2

3.1 Opis i cel zadania

Celem niniejszego zadania jest zaprojektowanie i implementacja autorskiego systemu wnioskowania rozmytego (FIS) dla problemu wielokryterialnego, różnego od zagadnień omawianych wcześniej. Projekt ma na celu praktyczne wykorzystanie logiki rozmytej do modelowania procesów decyzyjnych, w których dane wejściowe są nieprecyzyjne lub subiektywne. Kluczowym elementem jest właściwy dobór zmiennych lingwistycznych, funkcji przynależności oraz sformułowanie bazy reguł pozwalającej na uzyskanie logicznie uzasadnionego wyniku (outputu) systemu.

3.2 Implementacja systemu oceny studenta

W drugim zadaniu postanowiłem stworzyć system, który wspomaga wystawianie ocen końcowych. Jest to problem idealnie nadający się do logiki rozmytej, ponieważ pojęcia takie jak „dobra frekwencja” czy „udany egzamin” nie są zerojedynkowe.

3.3 Proces projektowania systemu

Realizacja zadania przebiegała w kilku kluczowych krokach:

1. **Definicja wejść i wyjścia:** Przyjąłem trzy zmienne wejściowe (*Antecedents*): frekwencję, wyniki prac samodzielnych oraz ocenę z egzaminu (wszystkie w skali 0–100). Wyjściem (*Consequent*) jest ocena w skali 2.0–5.0.

2. **Dobór funkcji przynależności:** Ponownie zdecydowałem się na funkcje Gaussa. Dzięki nim system jest „pobłażliwy” i unika ostrych granic – np. wynik 49% z egzaminu nie odcina studenta drastycznie od szansy na zaliczenie, jeśli inne parametry są wysokie.

3. **Sformułowanie logiki (reguł):** Stworzyłem 5 reguł, które oddają rzeczywisty model oceniania. Przykładowo, reguła *r2* działa jako bezpiecznik (słaby egzamin to niska ocena), a reguła *r3* pozwala ambitnym studentom podciągnąć wynik dzięki świetnym pracom domowym.

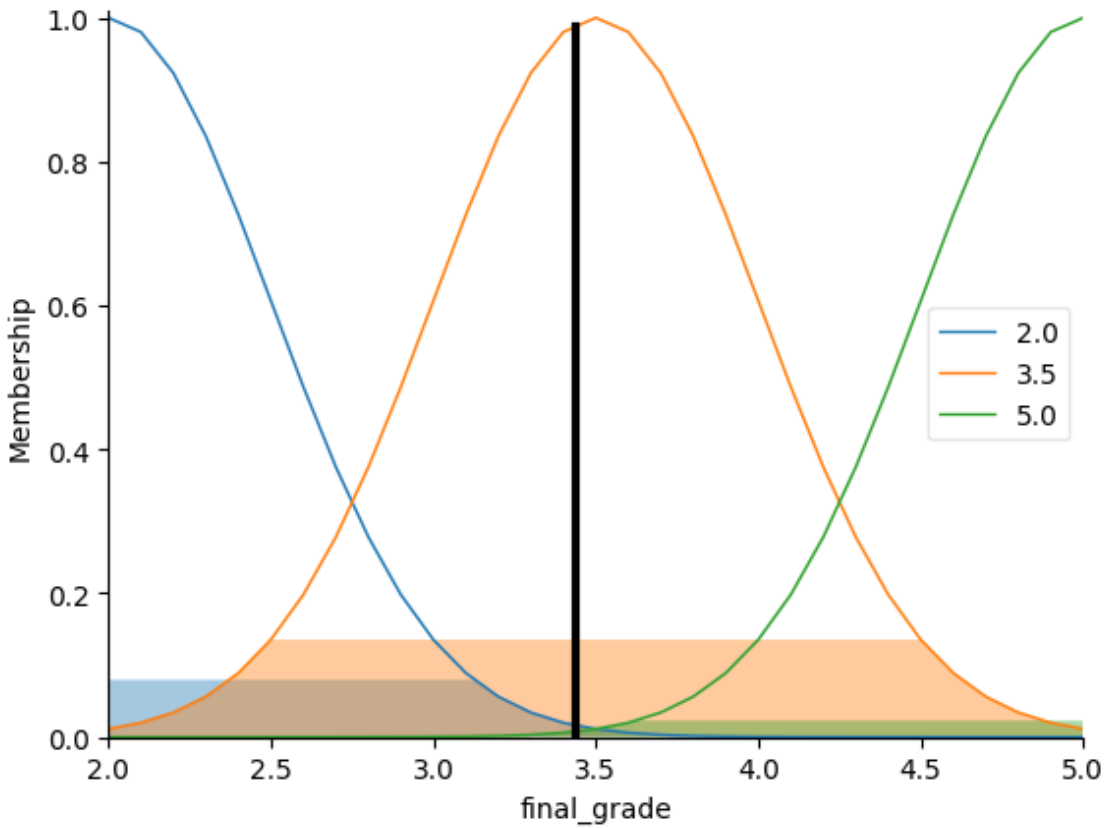
3.4 Przebieg wnioskowania i wynik symulacji

Aby sprawdzić system w praktyce, przeprowadziłem test dla studenta, który:

- miał bardzo wysoką frekwencję (90%),
- oddał przeciętne prace samodzielne (40%),
- napisał egzamin na poziomie 45% (pogranicze zaliczenia).

Wywołanie metody `grading_sim.compute()` uruchomiło proces fuzyfikacji danych, w którym system „zważył” wpływ każdej reguły. Wynik końcowy (np. 3.42) jest efektem defuzyfikacji metodą środka ciężkości (*centroid*).

3.5 Interpretacja wizualizacji



Rysunek 5: final_grade.view(sim=grading$_sim$)

`final_grade.view(sim=grading_sim)` jest kluczowy dla zrozumienia decyzji algorytmu:

- **Zaznaczony obszar:** Wypełniona bryła pod krzywymi funkcji przynależności oceny pokazuje, które reguły „odpaliły” i z jaką siłą. Widzimy tam wypadkowy kształt logiczny stworzony przez system.
- **Pionowa linia:** Cienka linia przechodząca przez bryłę wskazuje ostateczną, wyostrzoną wartość (*crisp value*). Jest to punkt równowagi (środek ciężkości) całego obszaru rozmytego.

Dzięki tej wizualizacji widać czarno na białym, że system nie wystawia oceny „z sufitu”, ale balansuje między różnymi wymaganiami, biorąc pod uwagę wszystkie mocne i słabe strony studenta.

4 Spis Literatury i odpowiednie linki

- Zbiór rozmyty